

Requirements Specification

Local Services Booking Platform

Project type: MERN web application

Intended organization: One company offering several local services

Version: Initial release (V1)

1. Project context and problem

People looking for local services, such as home cleaning, plumbing, or electrical repairs, often encounter offers spread across different channels. This makes it difficult to identify a suitable service, understand its details, and arrange an appointment in one place.

This project provides a website for one company that offers several types of local services. Customers can discover the company's services and submit appointment requests. A company administrator can maintain the catalog and handle reservation requests through a central interface.

2. Objectives

Main objective

Provide a clear online path from discovering a company service to requesting an appointment and following its status.

Customer objectives

- Browse the services offered by the company.
- Search and filter services, then read the details of a selected service.
- Request a reservation for a chosen date and time.
- View personal reservations and cancel one when needed.

Company objectives

- Create, edit, and remove service listings and organize them by category.
- Review reservation requests in one place and update their status.
- Consult customer account information needed for administration.

3. Users and roles

Actor	Access and responsibility
Visitor	Browse, search, and filter services; read service details; register or sign in.
Customer (user)	Use visitor features; submit a reservation request; view and cancel personal reservations.
Company administrator (admin)	Manage service listings and categories; view customers and reservations; update reservation statuses.

The administrator acts on behalf of the company. Staff who perform services have no accounts or assignments in V1. The application records customer requests and administrative decisions; internal staff coordination remains the company's responsibility.

4. Scope

Included in V1

- Customer registration, sign-in, and sign-out.
- Public service catalog with search, filters, and service details.
- Reservation requests and a personal reservation view.
- Administrator views for services, categories, reservations, and users.
- Input validation, access control, error handling, and essential tests.
- A deployable MERN application with documentation and UML diagrams.

Outside V1

Online payment, provider accounts, individual staff schedules or assignments, automated availability management, real-time chat, push notifications, reviews, and advanced geolocation are excluded. A requested date and time are not a guaranteed appointment until the company confirms the request.

5. Functional requirements

5.1 Accounts and access

- A visitor can create a customer account with a name, email address, and password.
- A customer can sign in, access private pages, and sign out.
- Only an administrator can access administrative pages and operations.
- Customers can access their own reservations; they cannot access another customer's reservation data.

5.2 Service catalog

- Visitors can see the service list and open a service detail page.
- Visitors can search services by name and filter them by category and city. Price filtering may also be offered.
- A service presents its title, description, price, city, and category; an image is optional.
- An administrator can create, edit, and remove services and manage categories.

5.3 Reservation requests

- A signed-in customer can select a service, date, and time and submit a reservation request.
- A new request starts with the status pending.
- A customer can view personal reservations and cancel a reservation.
- An administrator can view all reservations and set a status to pending, confirmed, completed, or cancelled.
- A confirmed status represents the company's acceptance of the request. A cancelled status represents a request that will not be fulfilled.
- The system rejects invalid dates and times and a duplicate request for the same service, date, and time when an active reservation already occupies that slot.

5.4 Administration

- An administrator can consult the customer list and basic customer details.
- Administrative actions on services, categories, users, and reservations require the admin role.

6. Rules, constraints, and quality requirements

Area	Requirement
Data validation	Reject missing required fields, malformed email addresses, invalid prices, and invalid reservation dates or times.
Security	Store passwords as hashes, authenticate private requests, enforce role permissions, and keep secrets outside the source repository.
Data privacy	Do not expose password hashes in API responses; restrict personal reservations to their owner and administrators.
Error handling	Return clear responses for invalid requests, unauthenticated access, forbidden actions, missing records, and server errors.
Usability	Keep the customer path understandable from search to request submission; show reservation status clearly.
Maintainability	Separate the React client, Express API, and MongoDB data layer; document setup and operation.
Project constraint	Keep V1 small enough to complete, test, deploy, and demonstrate within the project's planned three-week schedule.

7. User journeys and interface

Customer journey

Browse services -> Search or filter -> Read details -> Sign in or register -> Request a date and time -> View reservation status

The customer-facing pages include home, service list, service details, registration, sign-in, profile, reservation form, and personal reservations.

Administrator journey

Sign in -> Open dashboard -> Maintain services and categories -> Review requests -> Update reservation status

The administrator interface includes a dashboard and pages for services, categories, reservations, and users.

8. Technical overview

The browser runs a React frontend. It communicates over HTTP with an Express REST API. The API validates requests, applies authentication and role checks, and stores data in MongoDB through Mongoose.

React client -> Express API -> MongoDB

The main data entities are User, Category, Service, and Reservation. A service belongs to a category. A reservation links one customer to one service and stores the requested date, time, and status.

The planned supporting tools include JWT for authentication, bcrypt for password hashing, Jest and Supertest for backend tests, Docker Compose for local deployment, and GitHub Actions for continuous integration. The use-case diagram shows the actors and their actions, the class diagram shows the main data relationships, and the sequence diagram shows the login flow between the user, React frontend, Express API, and MongoDB.

8.1 Use-case diagram

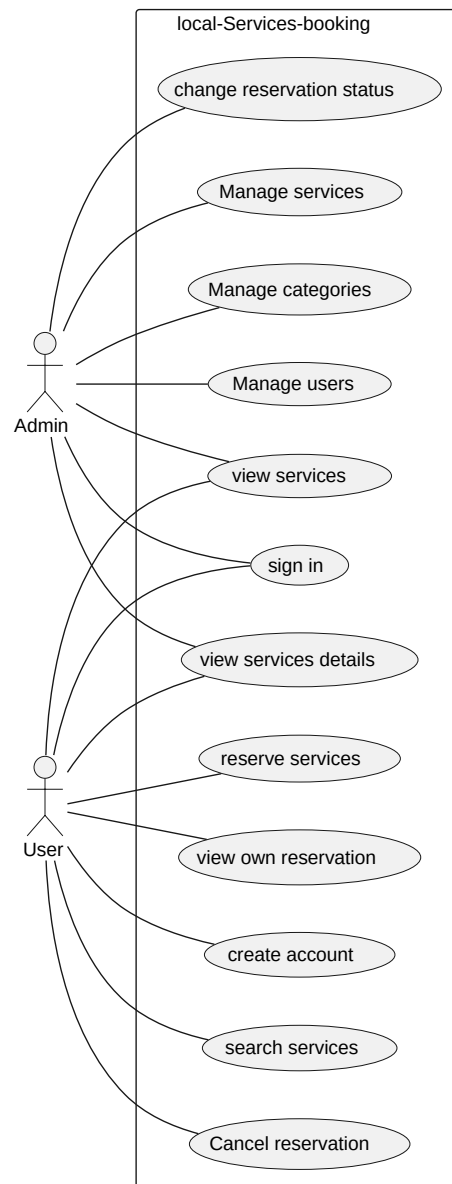


Figure 1. Customer and administrator actions within the application.

8.2 Class diagram

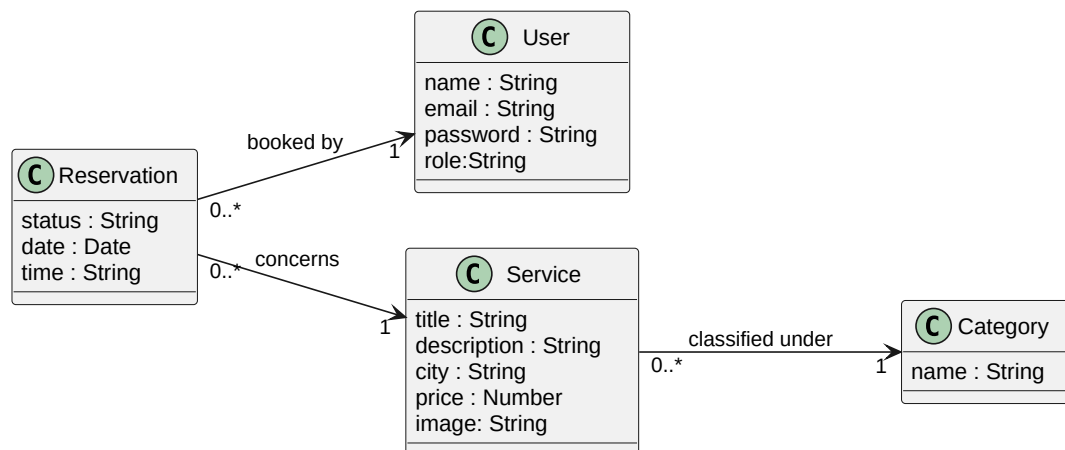


Figure 2. Main entities and their relationships.

8.3 Sequence diagram: Login

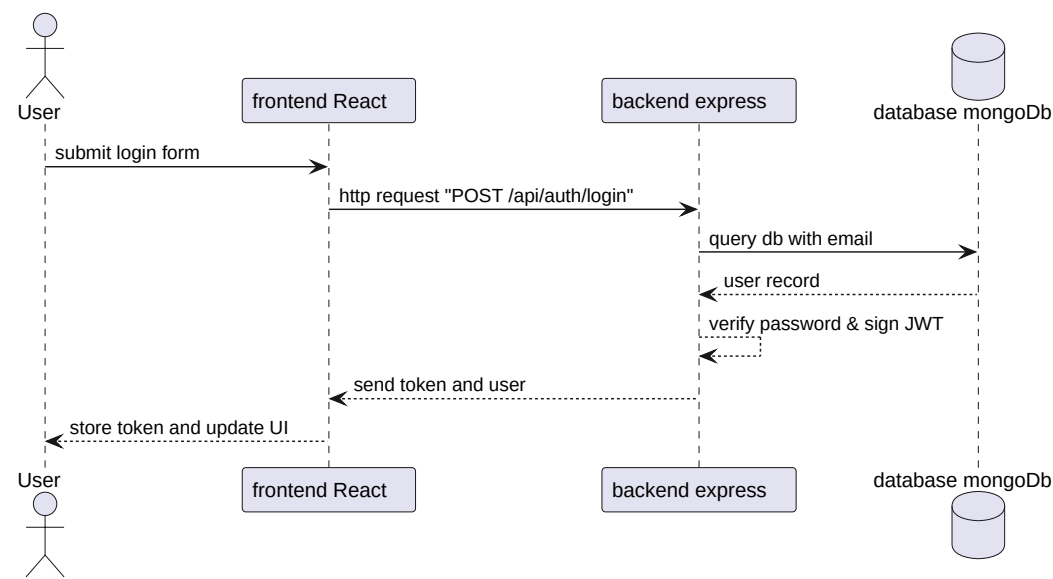


Figure 3. Login interactions between the user, React frontend, Express API, and MongoDB.

9. Acceptance criteria and deliverables

The V1 customer journey is accepted when a new customer can register, sign in, find a service through browsing or search, inspect its details, request a future date and time, and see the request in the personal area. An attempt to reserve an invalid or occupied slot must receive a clear rejection.

The administrator journey is accepted when an administrator can sign in, create or update a service, manage categories, view reservations, and change a reservation from pending to an appropriate decision status. A regular customer must be unable to perform those administrative actions.

The final project deliverables are:

- This requirements specification as a PDF.
- The React frontend, Express backend, and MongoDB data model in a GitHub repository.
- Use-case, class, and login sequence diagrams.
- Backend tests and a working demonstration of the principal journeys.
- Docker configuration, a GitHub Actions pipeline, a README, and deployment links when deployment is complete.

10. Project schedule

Phase	Planned result
Week 1	Backend foundation, data models, authentication, catalog, and reservations.
Week 2	Customer frontend journey and administrator interface.
Week 3	Tests, error handling, Docker, CI, deployment, UML, documentation, and demonstration preparation.

The essential customer and administrator journeys take priority over optional enhancements.